JustIT

Valerio Mazza · Giulio Rustia

Ingegneria del Software e Progettazione Web

University of Rome Tor Vergata - Department of Civil Engineering and Computer Science Engineering

Academic Year: 2025/2026

Contents

1. Software Requirement Specification

1.1. Introduction

1.1.1. Aim of the document

The purpose of this document is to present the fundamental and specific requirements of the "JustIT" system, a project for the "ISPW" Academic Year of 2025-2026

The document provides a comprehensive overview of the system's objective core functionalities and the rationale behind the design choices, in order to guide the development process and ensure alignment among all team members.

Team members: Valerio Mazza, Giulio Rustia

1.1.2. Overview of the defined system

The system is a Java-based software platform structured according to the MVC architecture developed with Maven, and featuring two user interfaces:

- JavaFX and FXML files.
- Standard Command Line Interface.

The system enables efficient booking and appointment management for technical shops. Features included are:

- Management of clients and technicians.
- Management of technical shops.
- Browsing and searching technical shops near you.
- A system to review the service.
- View management of the bookings and appointment
- Exporting a booking calendar on csv

The system supports two types of users, each with different permissions and functionalities:

- Technicians.
- Customers.

Furthermore, the application can be executed in two different modes:

- Demo-version: execution with in-memory persistence (volatile data).
- Full-version: execution with persistence on a relational database or file system.

The system supports two persistence modes:

- Relational SQLite database.
- File-System based on Json files.

1.1.3. HW e SW requirements

Software requirements

- Java 21+
- IDE: Intellij IDEA or Eclipse
- SQLite Database
- Operating System: Any Linux based distro with java 21+ support, Windows 10+, macOs 10.12+

Hardware Requirements

- CPU: Dual-Core 2.0 Ghz
- Ram: 2 GB

- Disk space: 1 GB

1.1.4. Related systems

1.1.4.1. TaskRabbit

Pro

- Extensive directory of professionals with category filters
- Robust review and reputation system
- Detailed requests with price quotes

Cons

- Workflow is too fragmented for quick bookings

1.1.4.2. ProntoPro

Pro

- Prioritizes local pros and small businesses
- Easy quote comparison
- Covering all of Italy

Cons

- Less emphasis on map/radius search

1.2. User Stories

1.2.1. User

- As a User, I want to find all shops near me, so that i can choose which one to call.
- As a User, I want to filter technicians by distance, so that I can receive support faster
- As a User, I want to rate and review my experience, so that I can share feedback on the shop's service quality

1.2.2. Technician

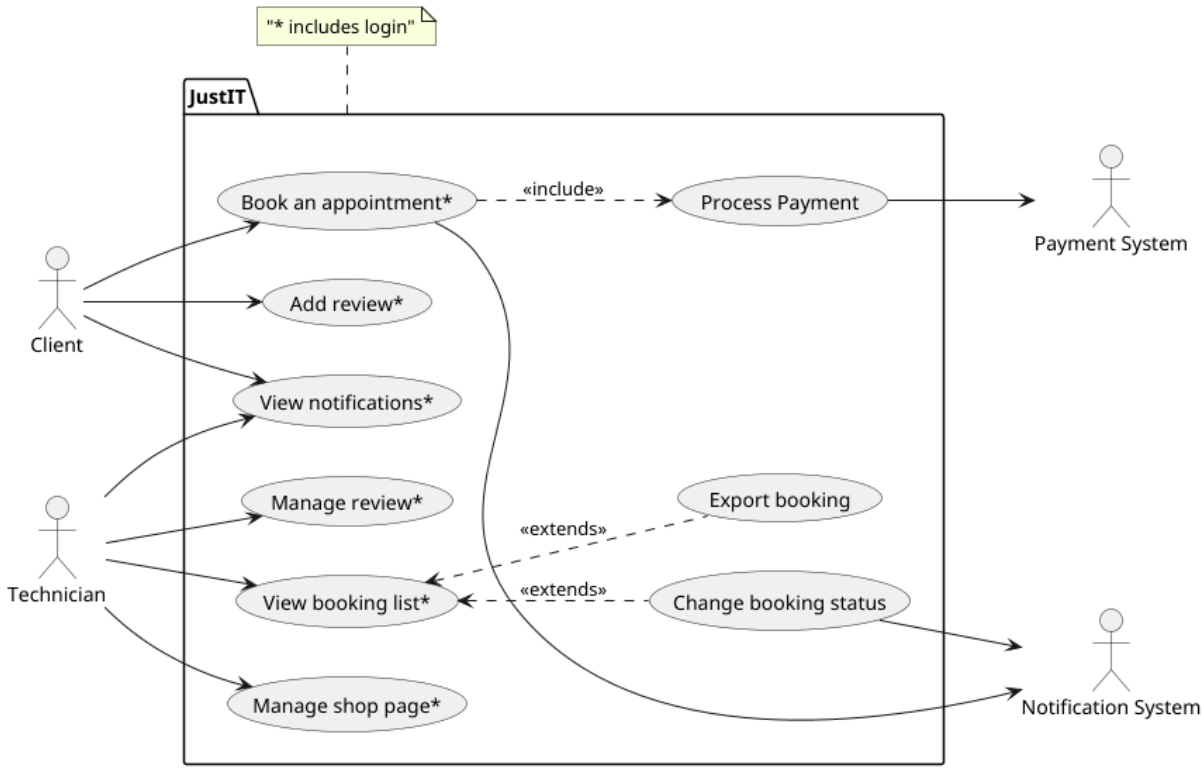
- As a Technician, I want to accept or reject appointments, so that I can ensure I only take jobs within my area of expertise.
- As a Technician, I want to list my service offerings, so that potential clients can easily view what I provide.
- As a Technician, I want to download my booking history, so that I can manage my agenda more efficiently.

1.3. Functional Requirements

- The system shall display the current status of a booking.
- The system shall notify the User of any booking status change
- The system shall provide a filter for Users to display shops located within a customizable radius from their address location
- The system shall display a detailed profile for each Shop, including services and reviews.
- The system shall provide the Technician's booking history in CSV format.
- The system shall provide a booking form for the User to describe the issue.

1.4. Use Cases

1.4.1. Use Cases Diagram



1.4.2. Use case internal steps

1.4.2.1. Use case: Book an appointment

Main flow:

- 1. The user accesses the “Browse Shop” section via Login.
- 2. The system displays a selection of nearby shops.
- 3. The User chooses a shop.
- 4. The system displays the shop page information.
- 5. The user selects “Book Now”.
- 6. The system shows a booking form.
- 7. The user selects the preferred time slot and confirms the selection.
- 8. The system processes the payment, records the booking and notifies the technician.

Extensions:

- 1a. The user fails the authentication.
- 1b. The system displays an error and allows them to retry.
- 7a. The user has already booked the selected shop.
- 7b. The system displays an error message and returns the user to the Shop page.
- 8a. The user fails to complete the payment within 5 minutes.
- 8b. The system display an error message and returns the user to the booking form.

1.4.2.2. Use case: Manage Booking Status

Main flow:

- 1. The technician accesses the “Booking List” section.
- 2. The system displays the list of bookings.

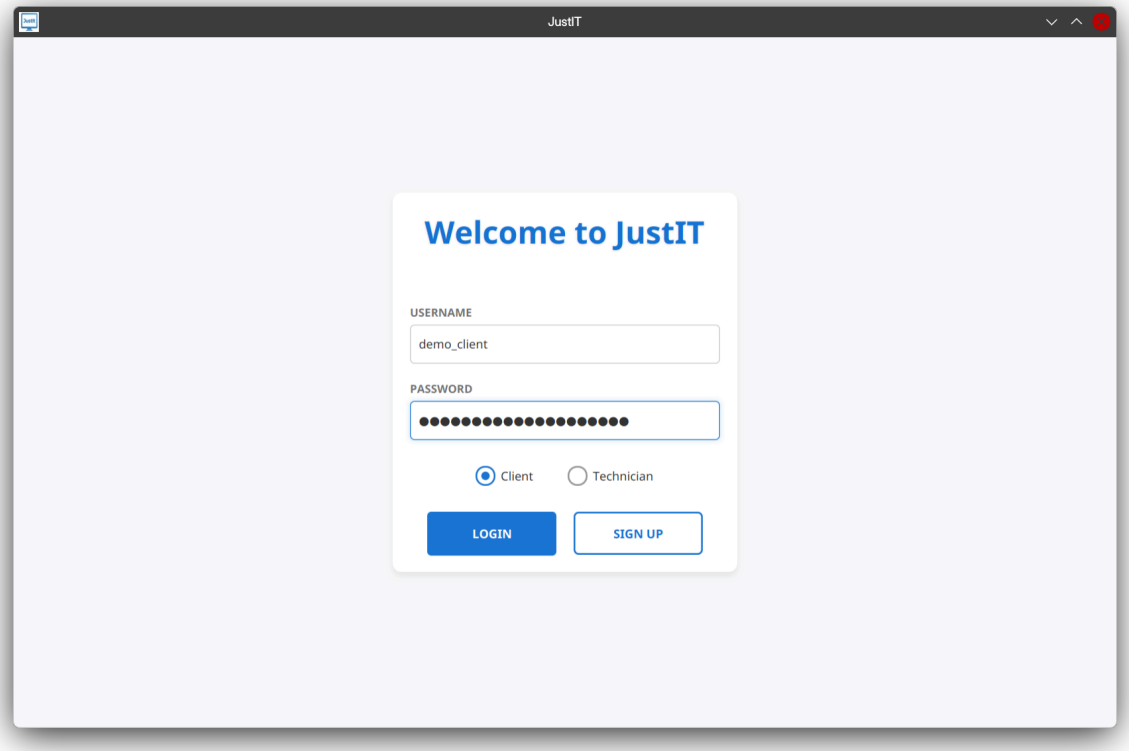
- 3. The technician selects a booking.
- 4. The system displays the details of booking.
- 5. The technician changes the status of the booking.
- 6. The system validates the requested status transition.
- 7. The system updates the booking status.
- 8. The system sends a notification to the user associated with the booking.

Extension:

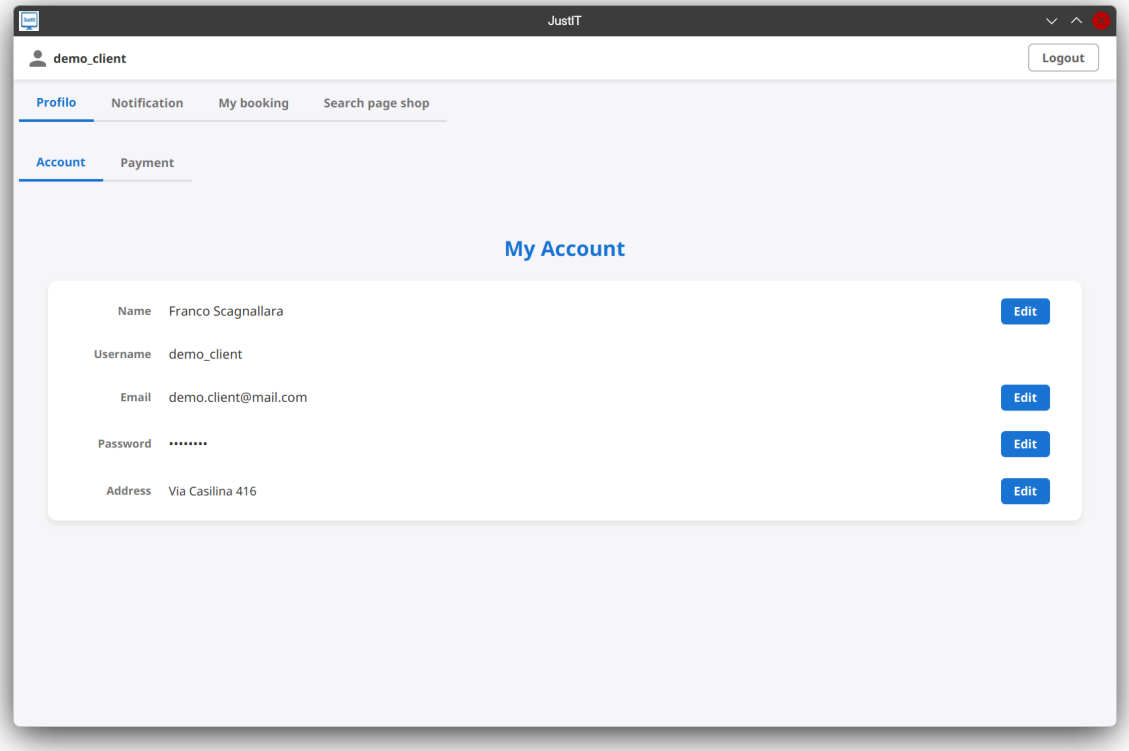
- 2a. The system detects that there are no bookings yet.
- 2b. The system displays a message informing the Technician that no bookings are available.
- 6a. The technician selects a status change that is not allowed.
- 6b. The system prevents the status update.
- 6c. The system displays an error message indicating that the selected transition is not valid.

2. Storyboards

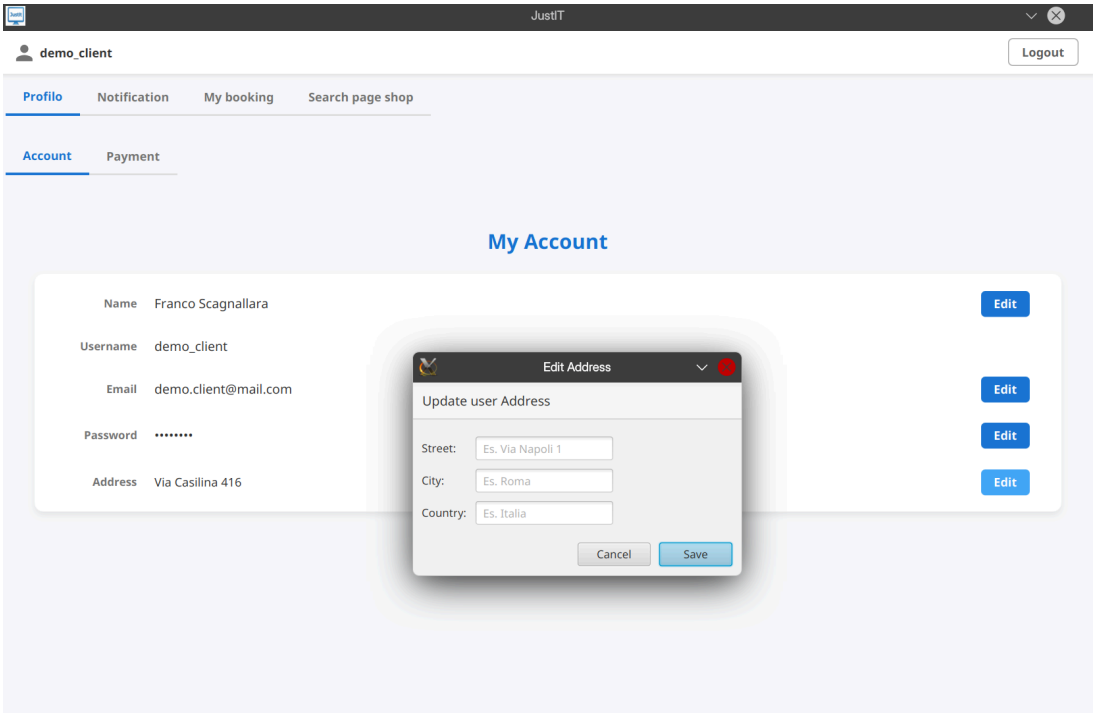
2.1. Login



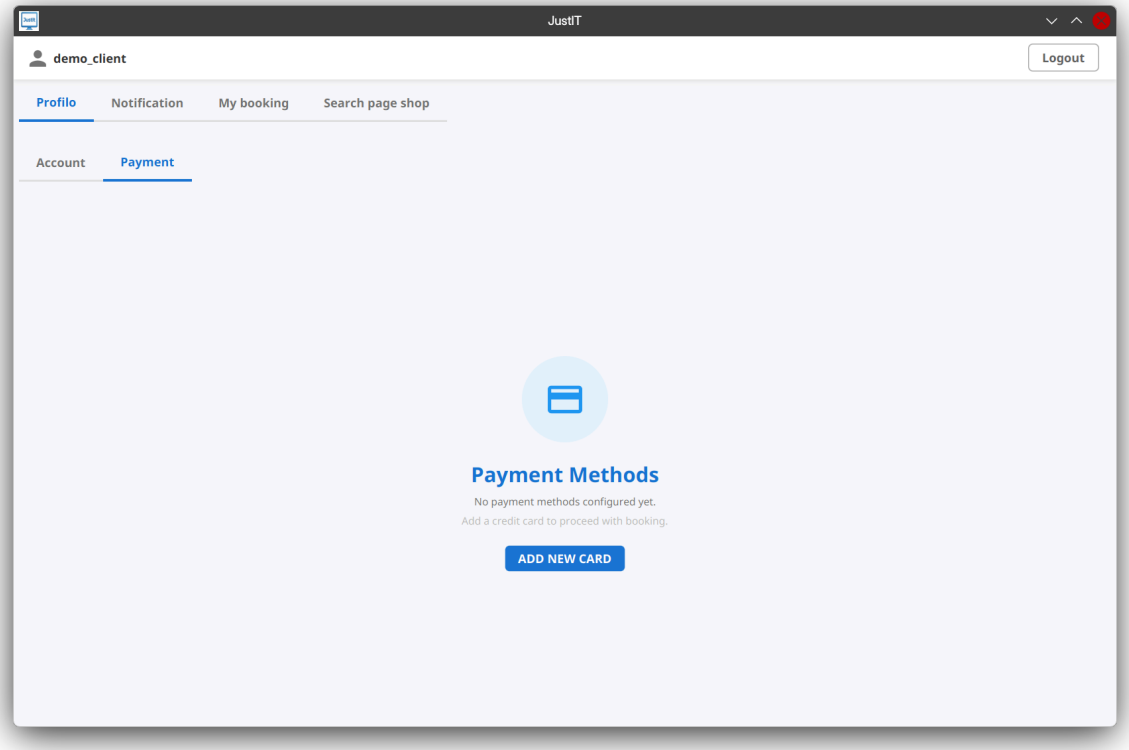
2.2. Account page client



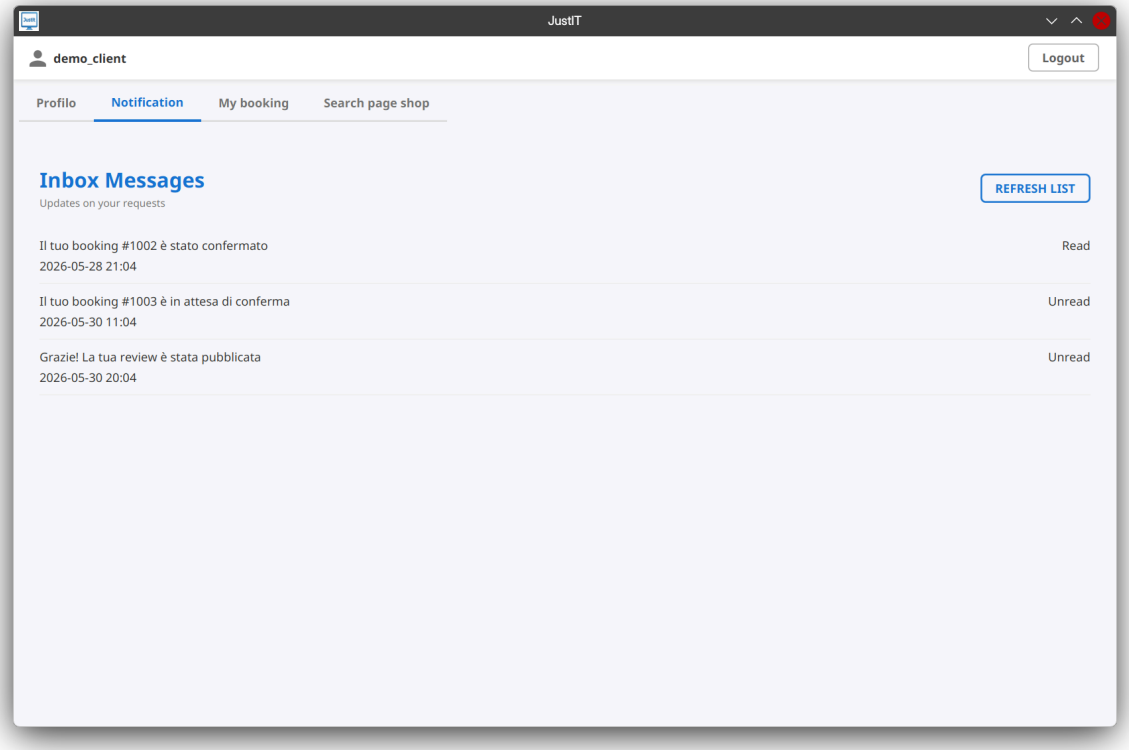
2.3. Update address



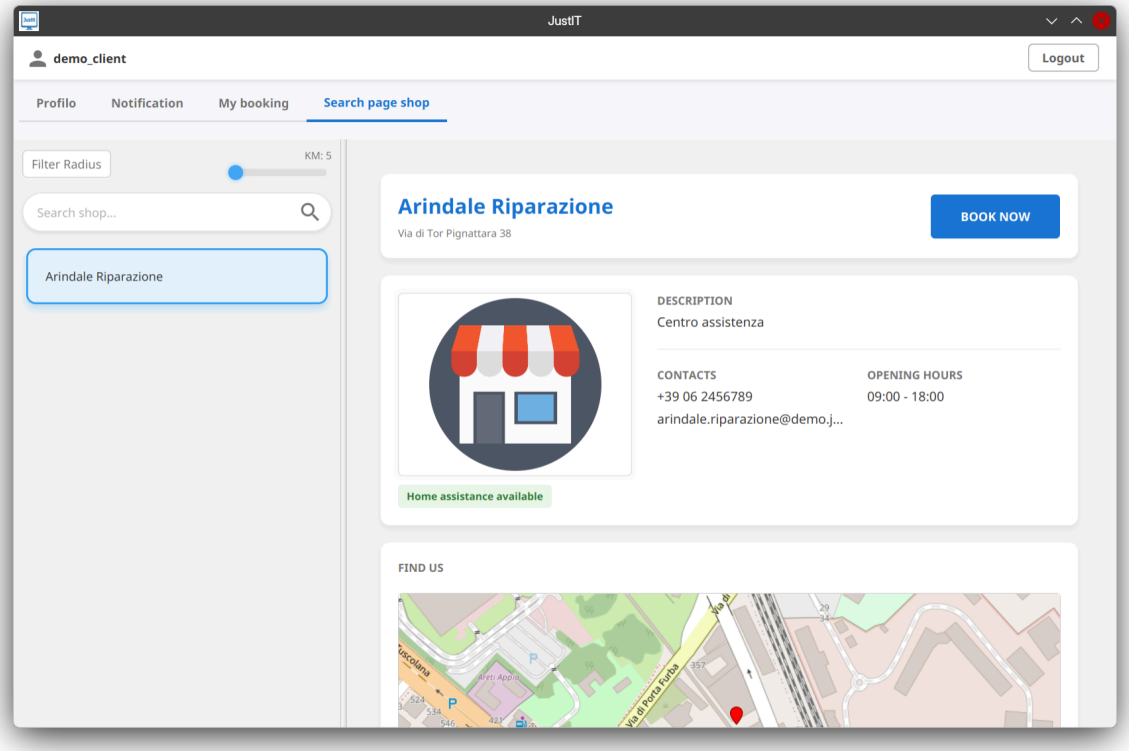
2.4. Add payment card



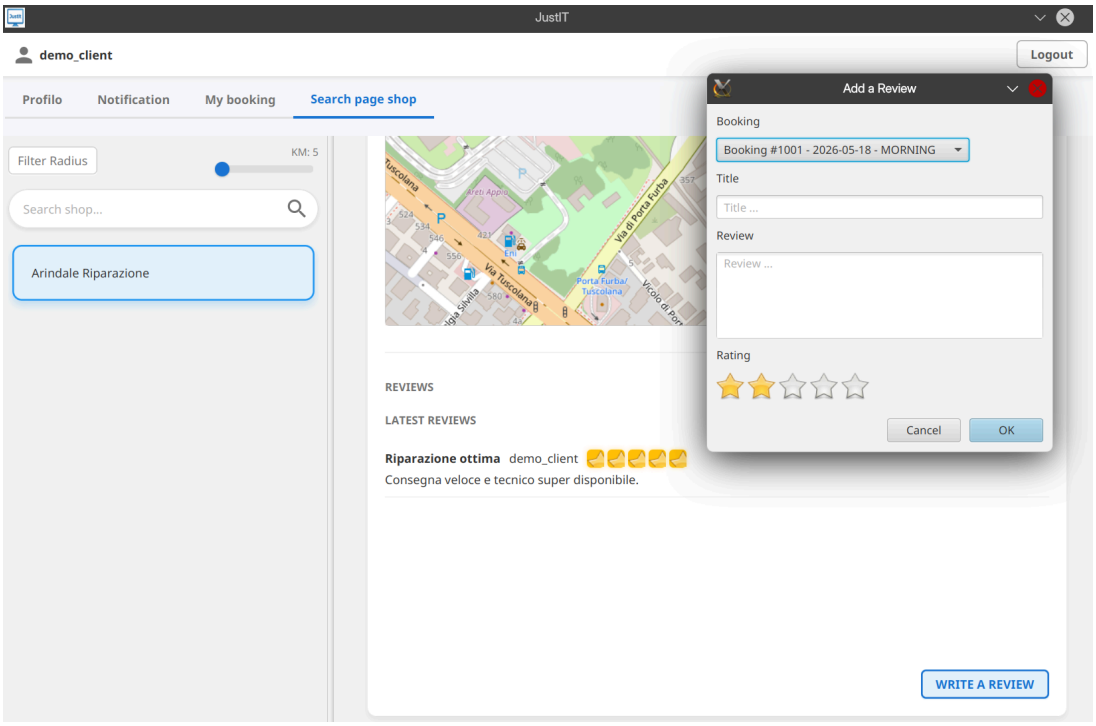
2.5. Inbox notification client



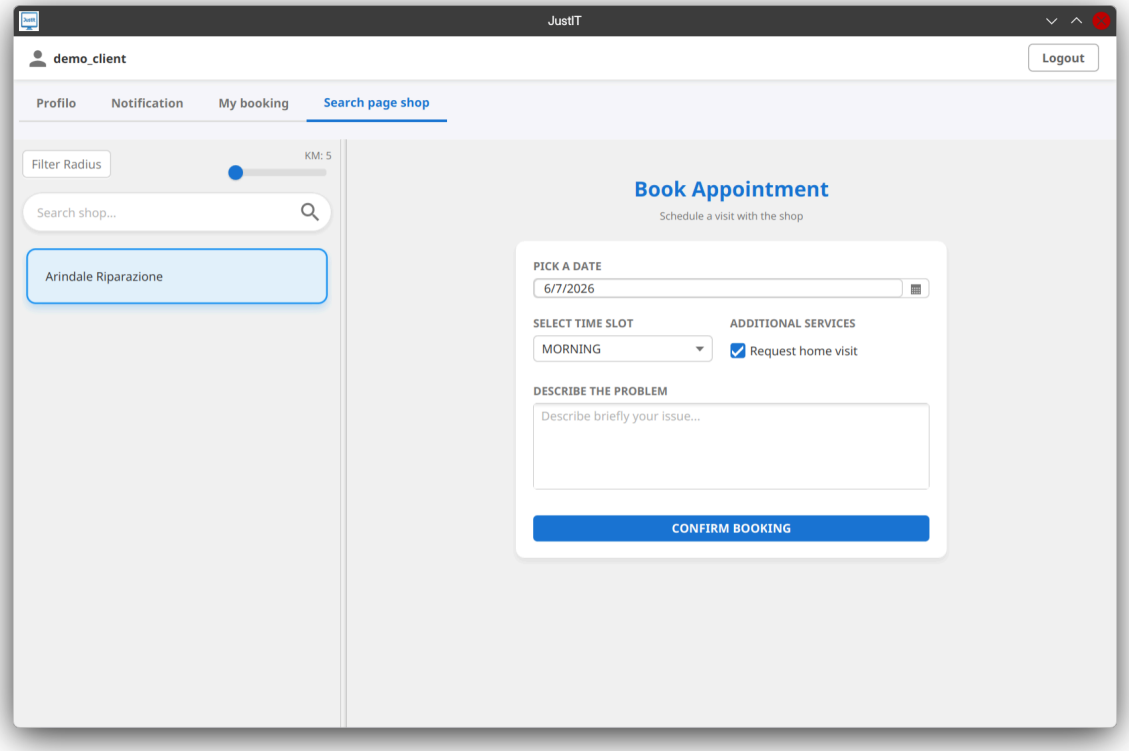
2.6. Search and Page shop



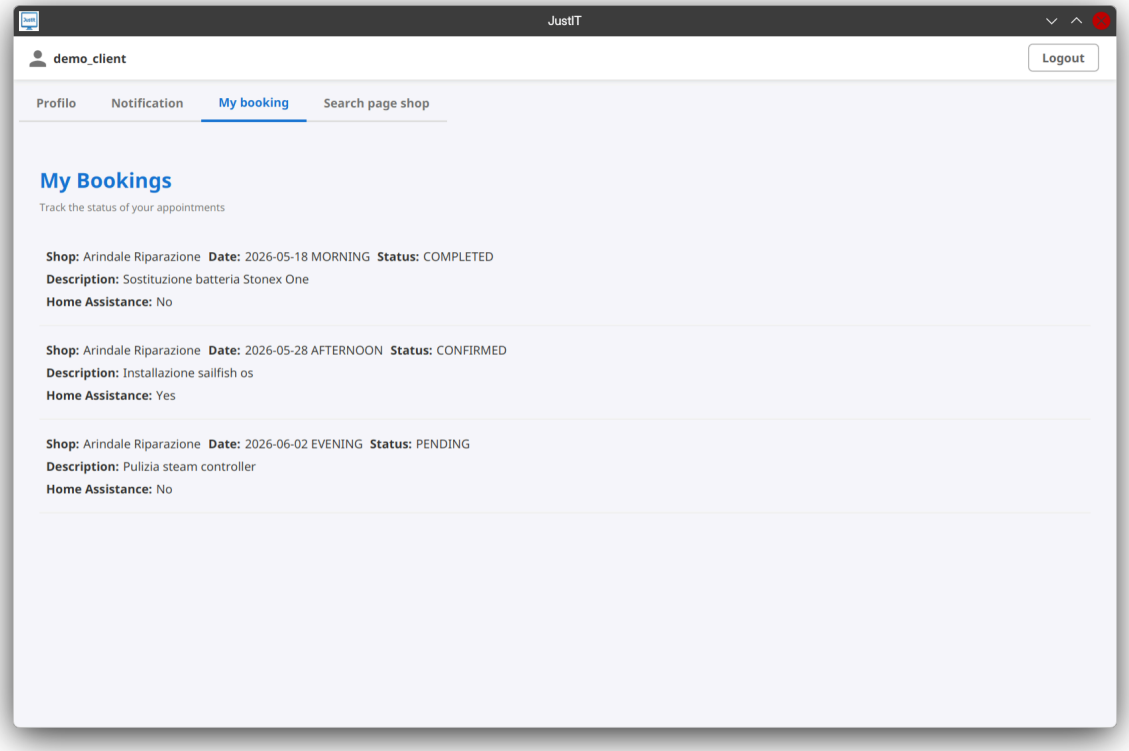
2.7. Write review



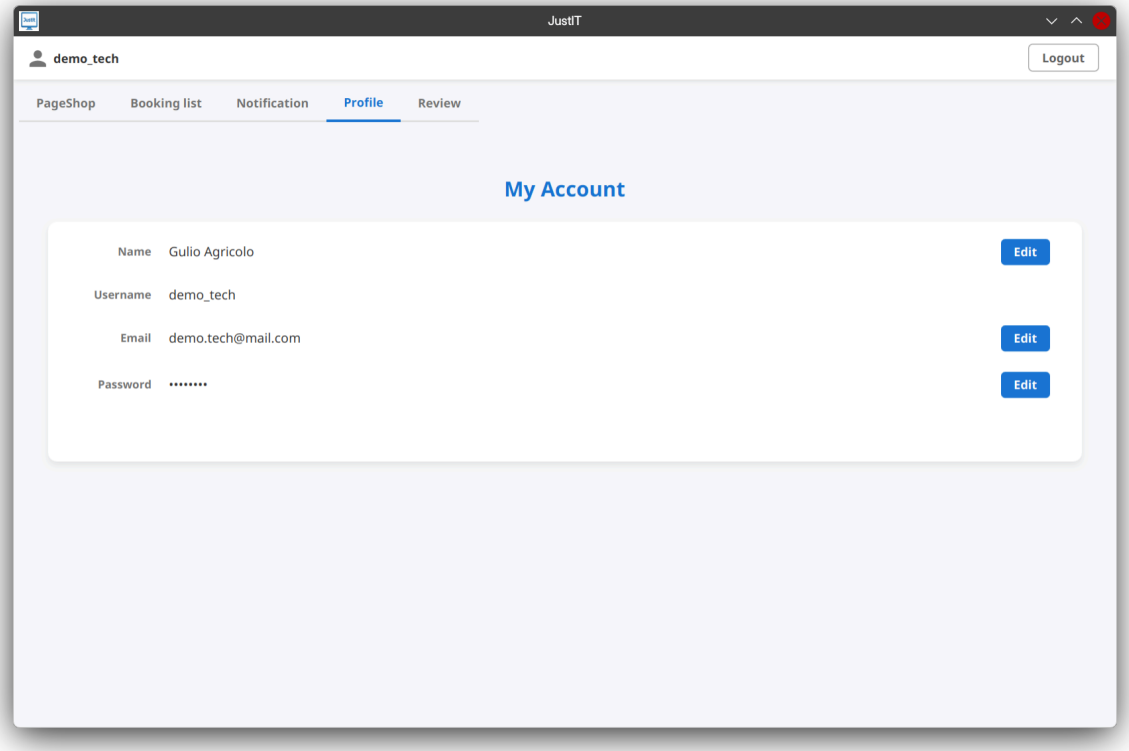
2.8. Booking



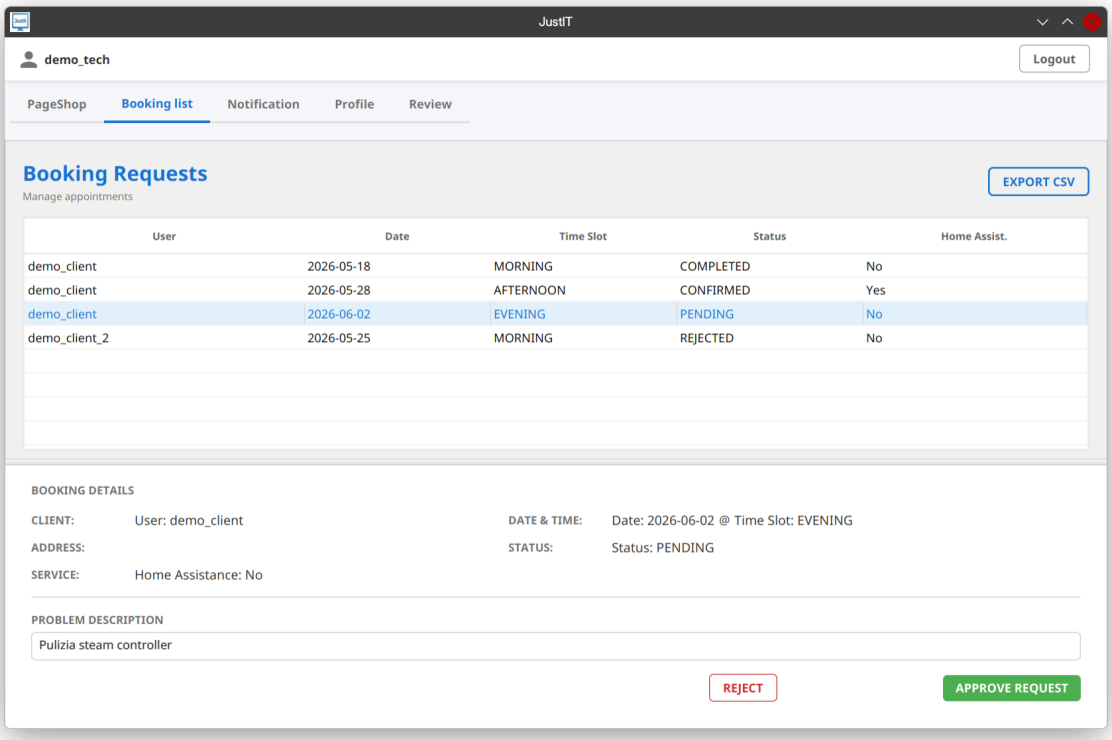
2.9. Booking list



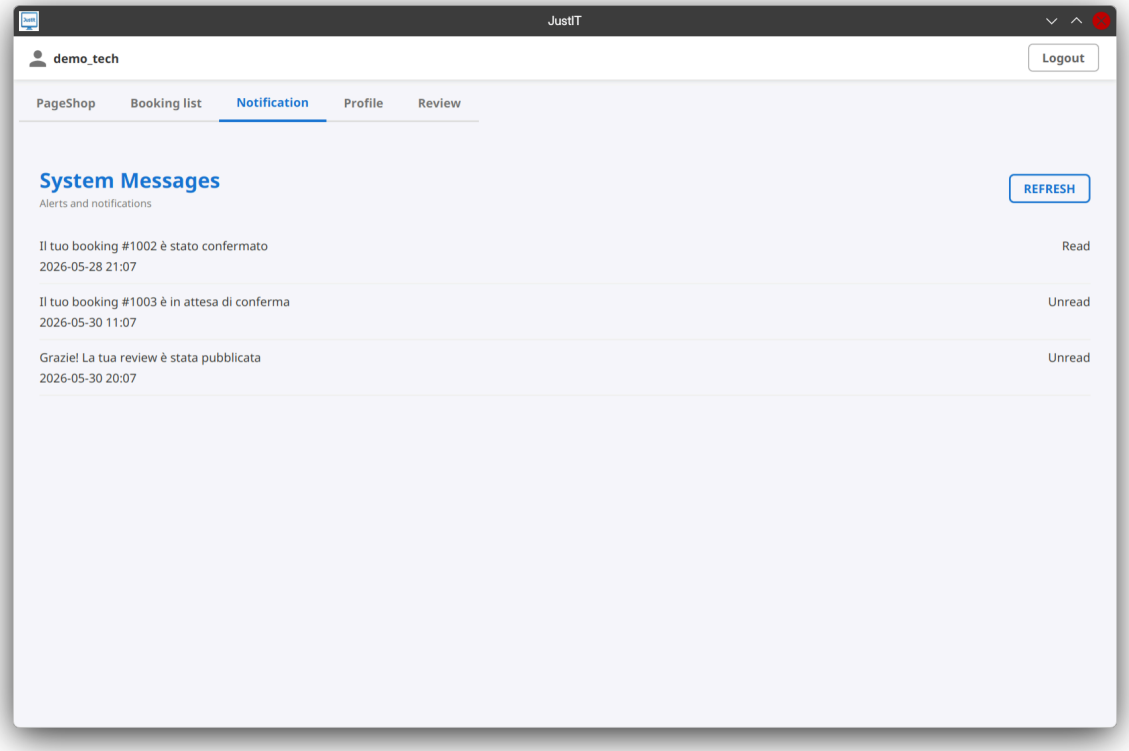
2.10. Account tech



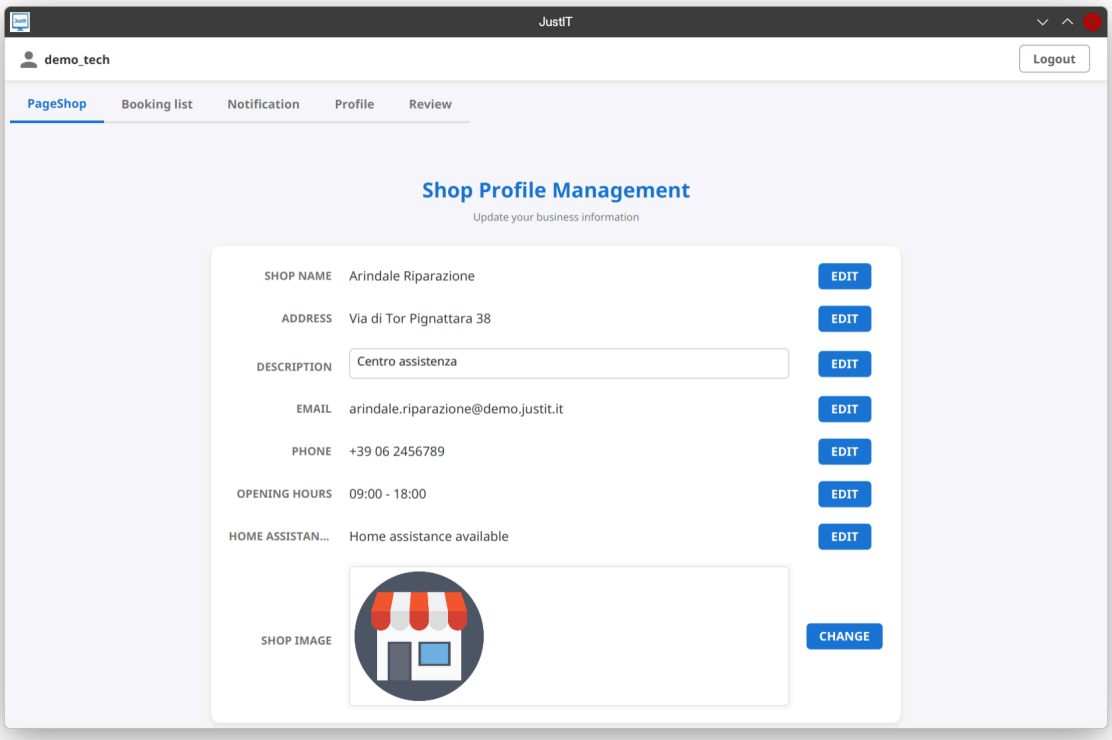
2.11. Manage bookings



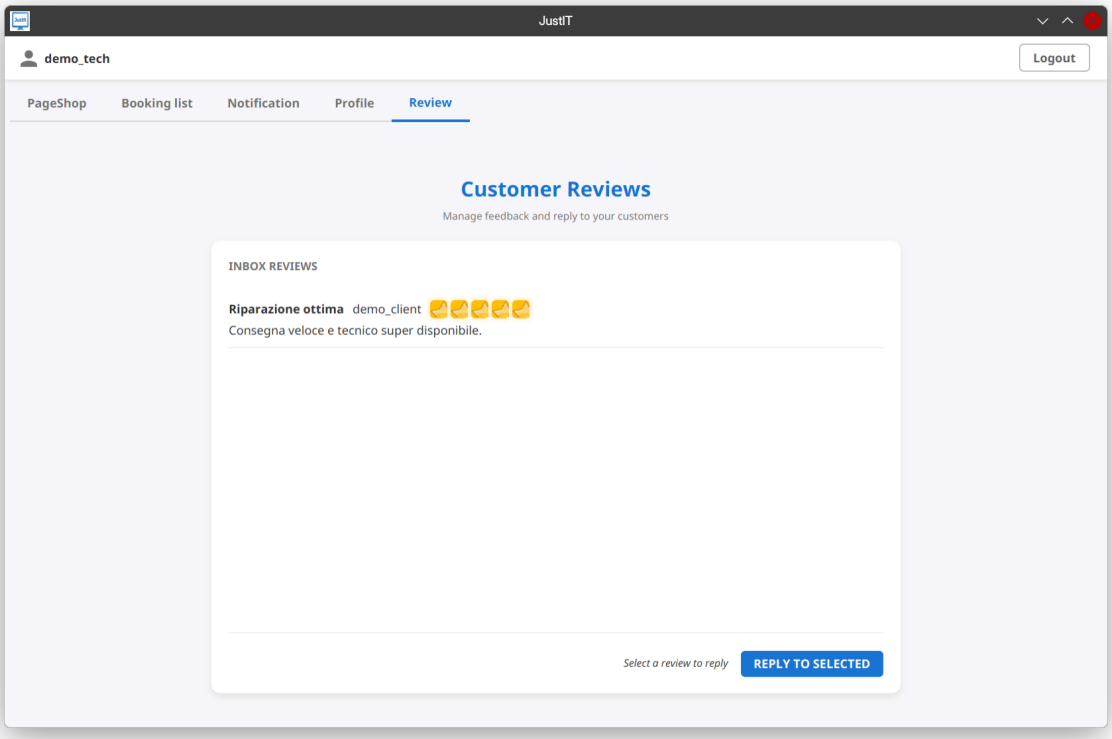
2.12. System notification



2.13. Manage page shop



2.14. Manage customer reviews



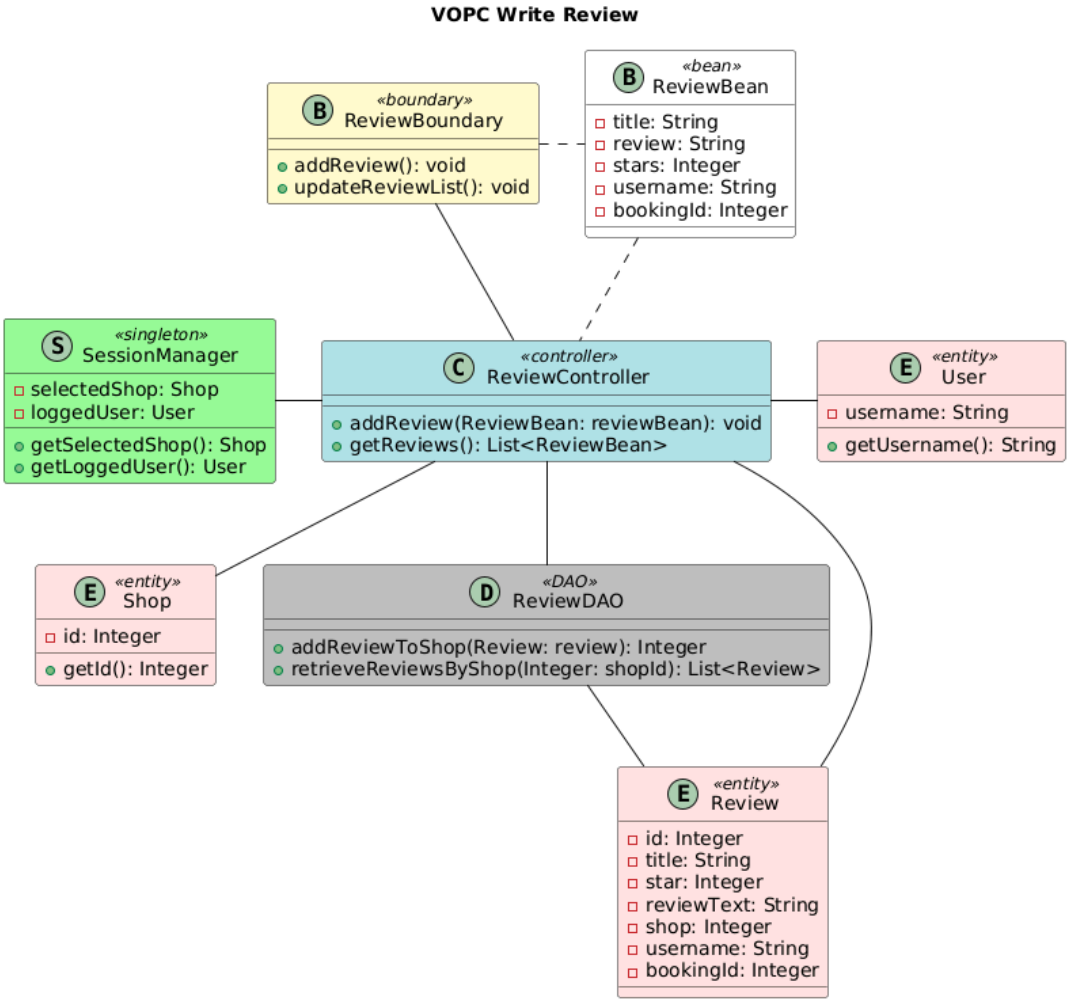
3. Design

Since design diagrams are created before development begins, they may not perfectly reflect the final codebase.

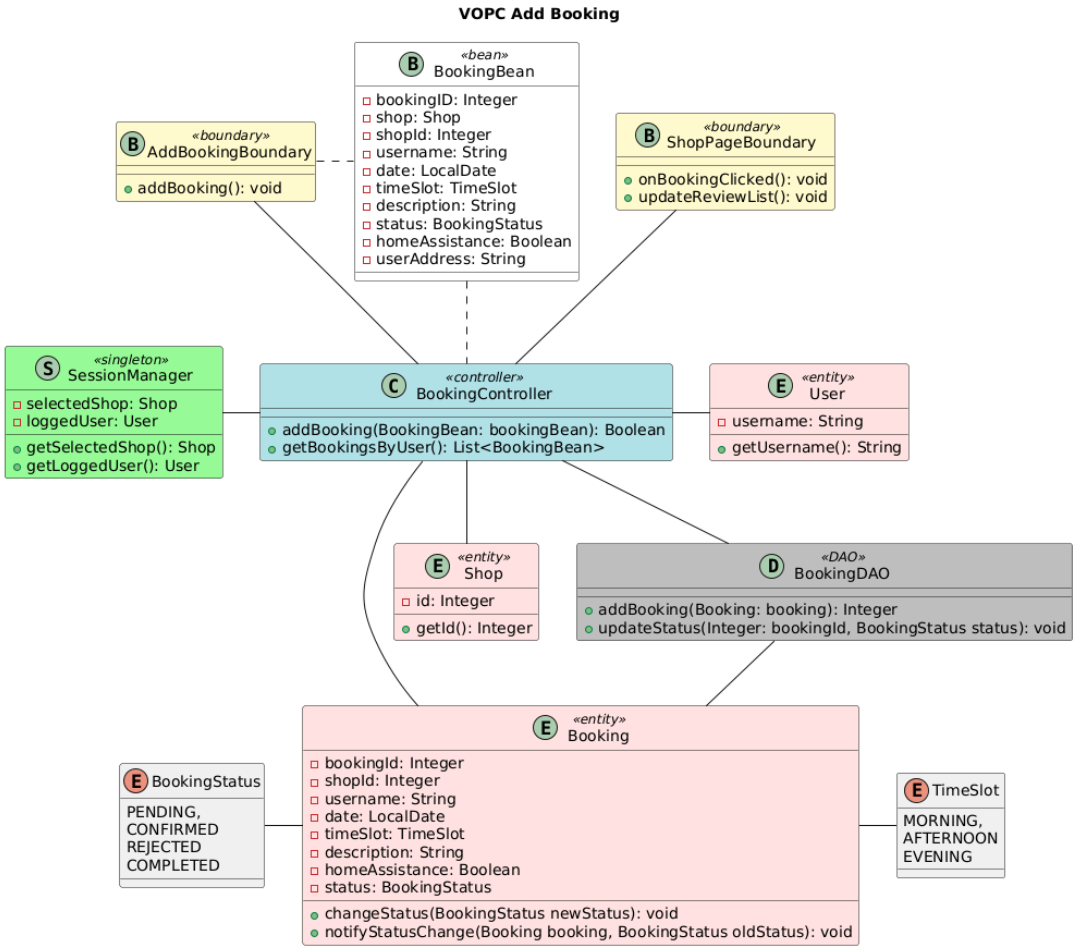
3.1. Class diagram

3.1.1. VOPC Analysis

3.1.1.1. Write review

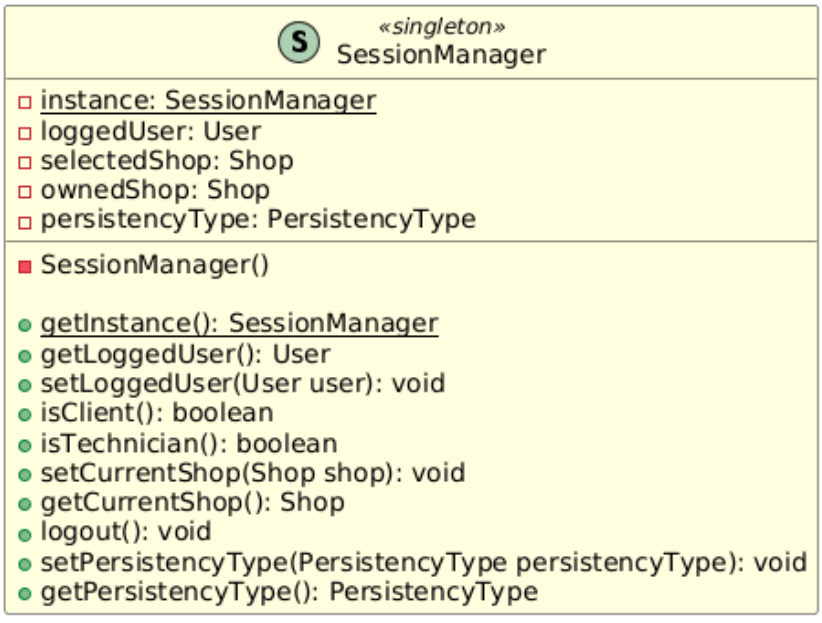


3.1.1.2. Add booking

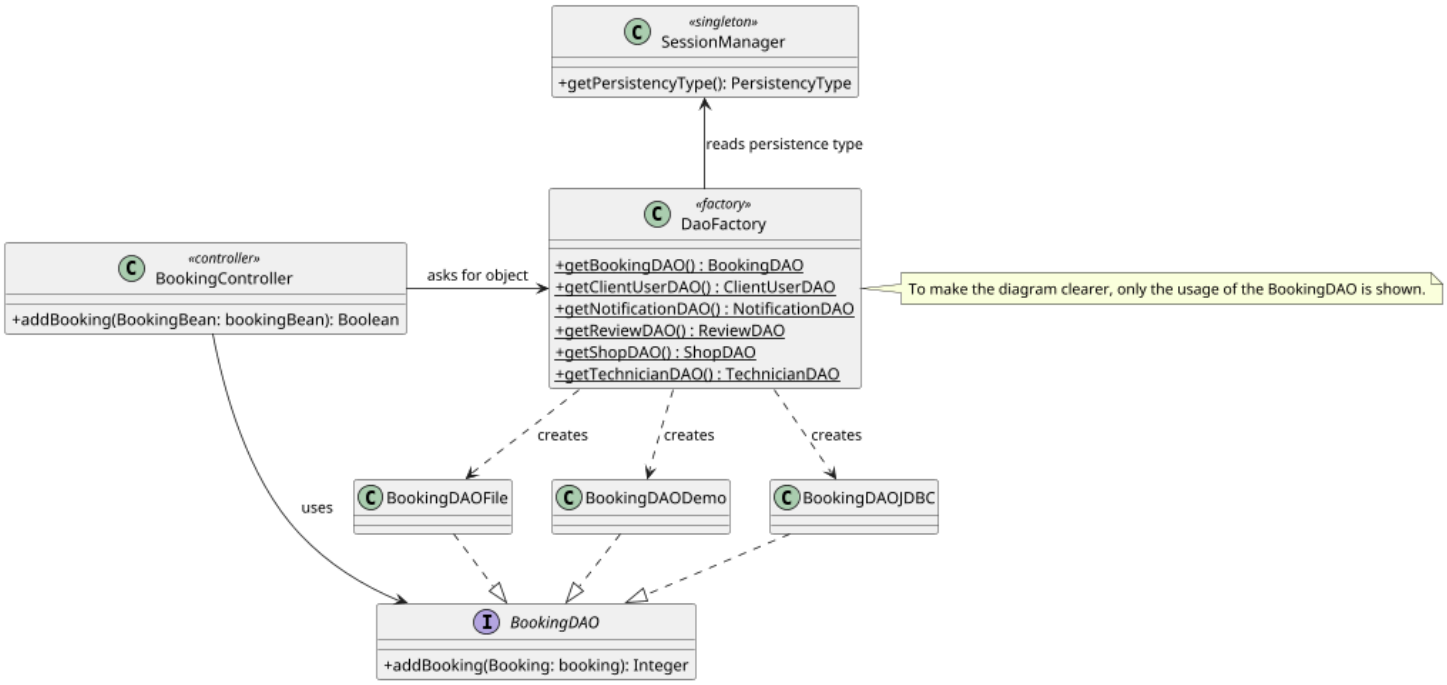


3.1.2. Design Level Diagram

3.1.2.1. Singleton



3.1.2.2. Factory Method



3.2. Design patterns

3.2.1. Singleton pattern

In JustIT, we've adopted the Singleton pattern to manage the currently logged user session.

The SessionManager acts as the single access point for session state and is invoked across the system via getInstance(), avoiding repetitive object passing in the system and allow consistency.

After login, controllers set the current user with setLoggedUser, making user data (getLoggedUser) and role checks (isClient/isTechnician) available everywhere.

Shop context is centralized through setCurrentShop and getCurrentShop: for clients it stores the shop selected during browsing, while for technicians it stores the owned shop, so booking, review, notification, and account controllers can consistently read shop ids and details without duplicating logic.

Finally, logout clears all shared state, ensuring the session restarts cleanly.

3.2.2. Factory Method Pattern

In JustIt, a centralized Factory pattern is adopted to manage the creation of all Data Access Objects (DAOs) through the DaoFactory class.

The factory is responsible for providing the appropriate concrete implementation for each DAO interface. The selection of the specific implementation is driven by the current persistence configuration retrieved from the SessionManager.

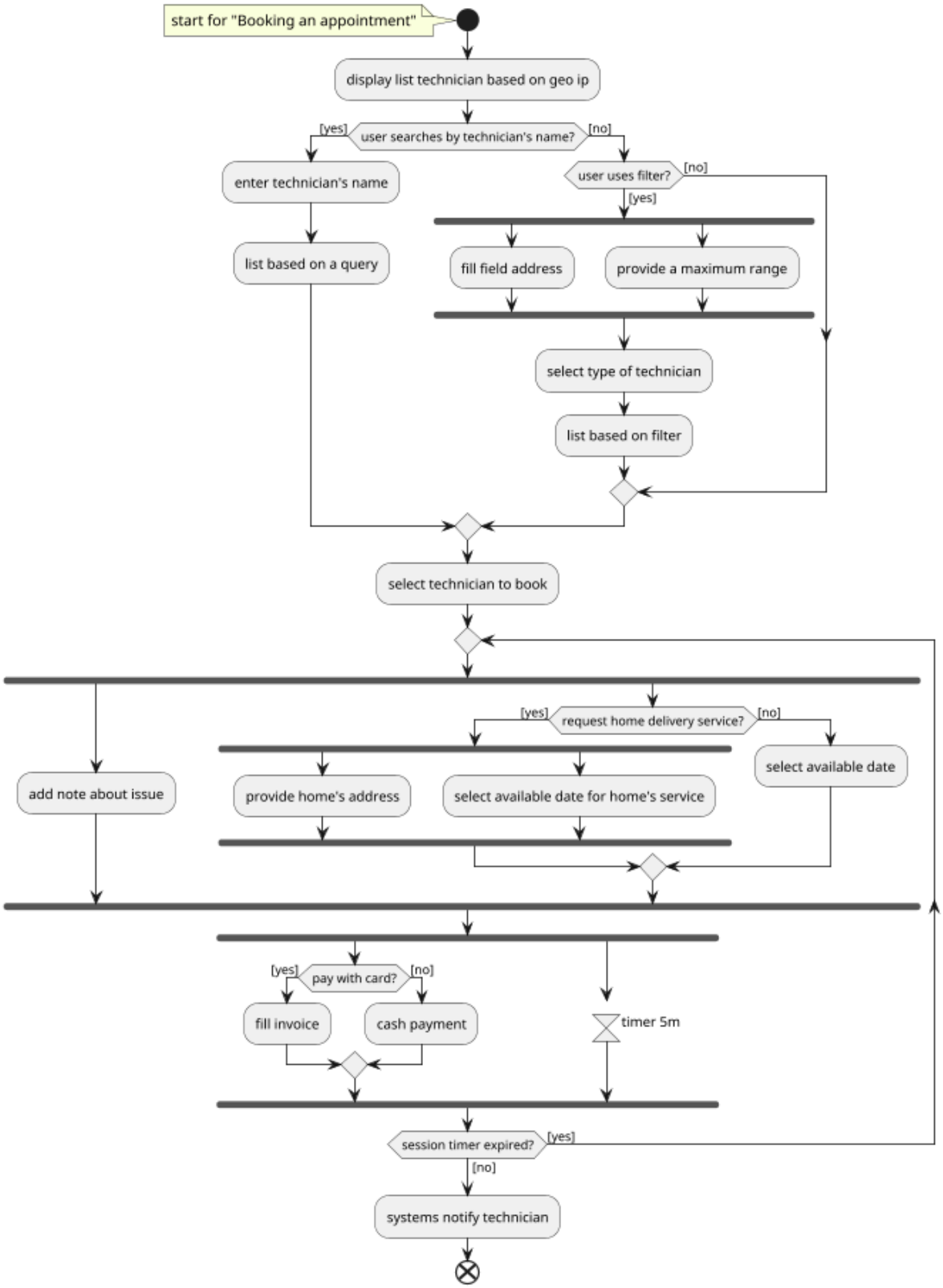
Each DAO interface defines a persistence contract, with concrete implementations (JDBC, FileSystem, Demo) providing alternative storage backends.

This design decouples the controller layer from all persistence implementations, allowing controllers to interact exclusively with DAO abstractions without knowledge of the underlying storage technology. As a result, the system can switch between different persistence modes without requiring changes in the business logic.

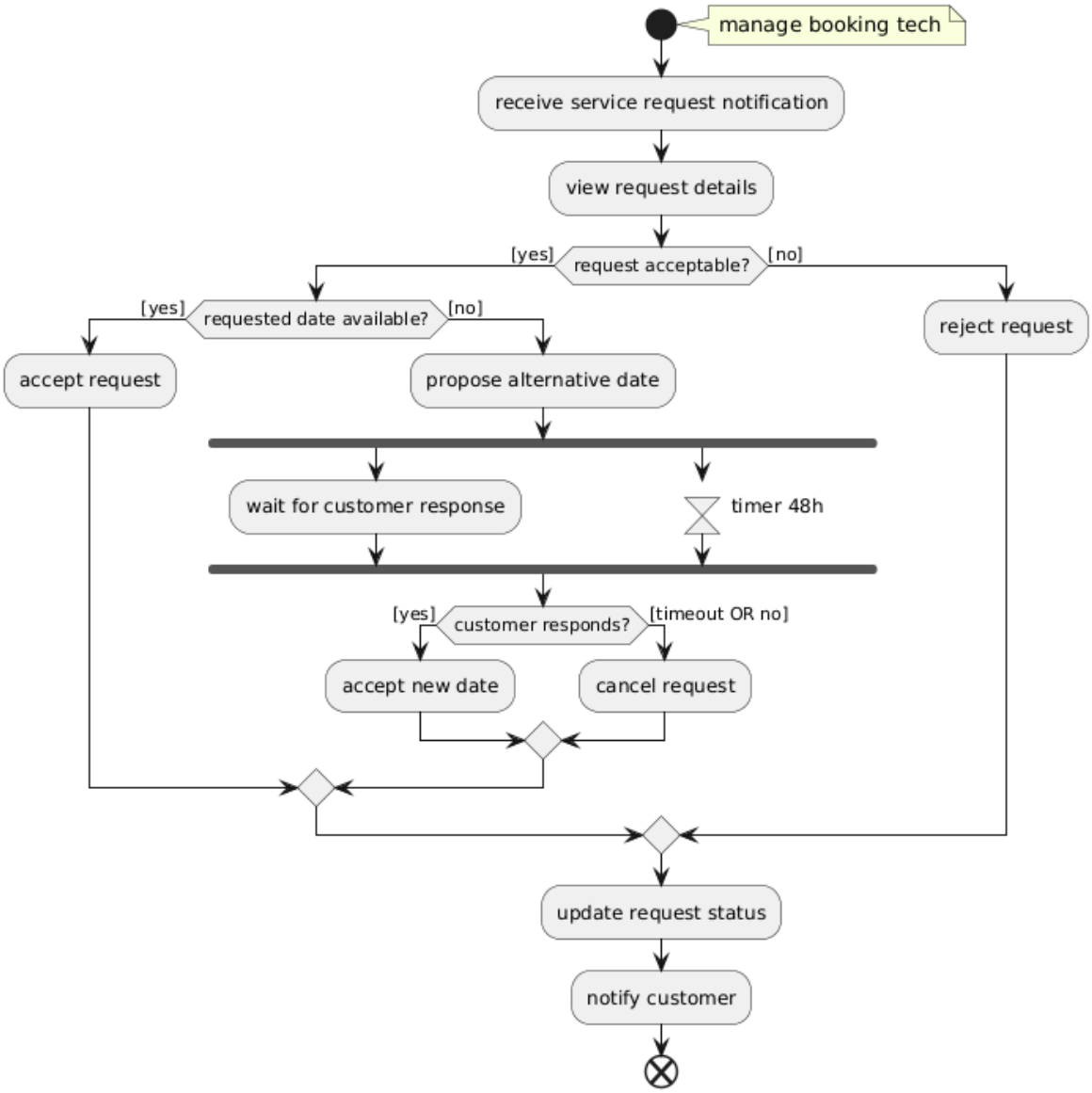
Centralizing object creation in the factory improves modularity, maintainability, and scalability, enabling the addition of new persistence strategies or DAO implementations with minimal impact on the existing codebase.

3.3. Activity Diagram

3.3.1. Booking an appointment

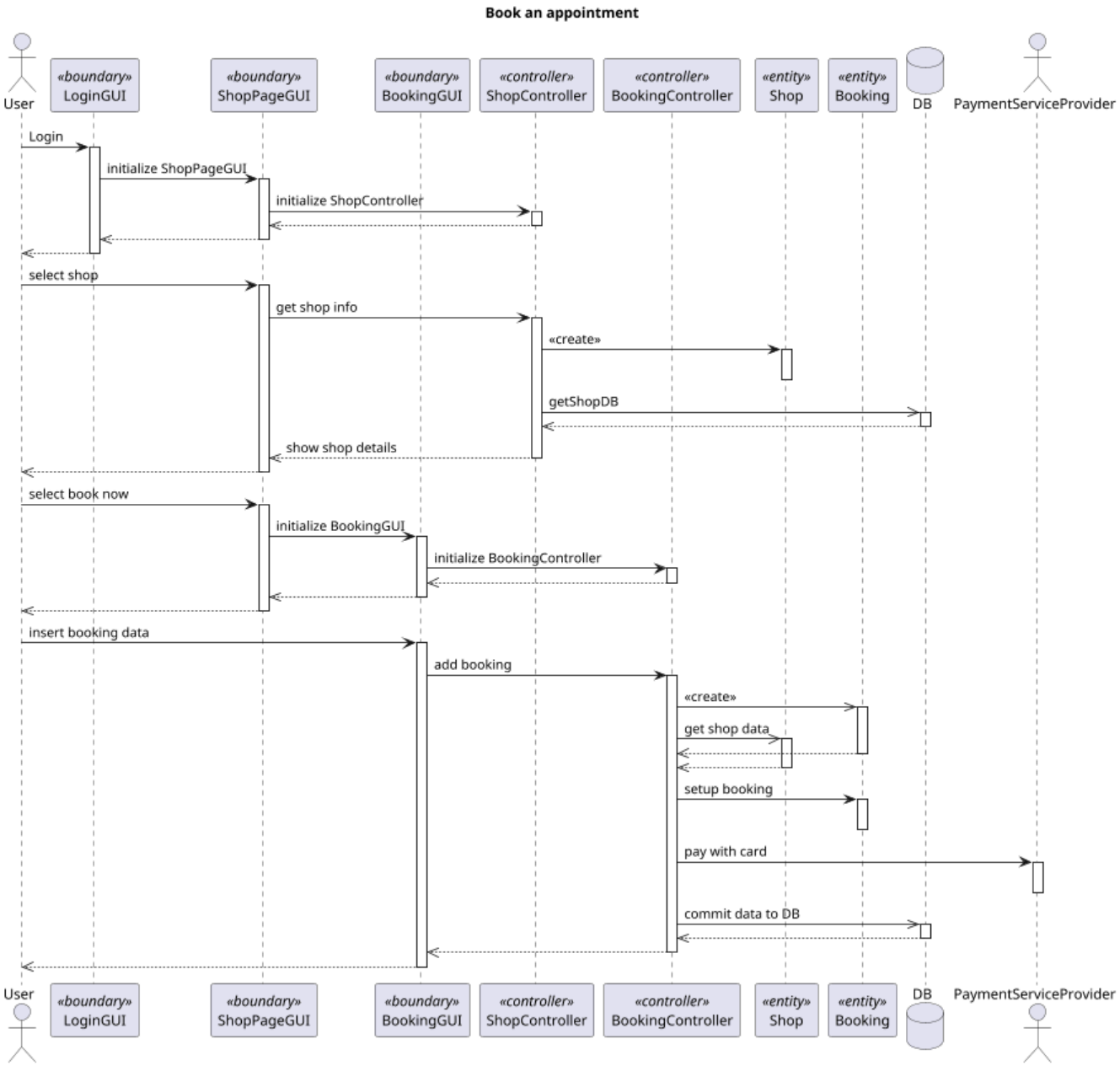


3.3.2. Manage Booking

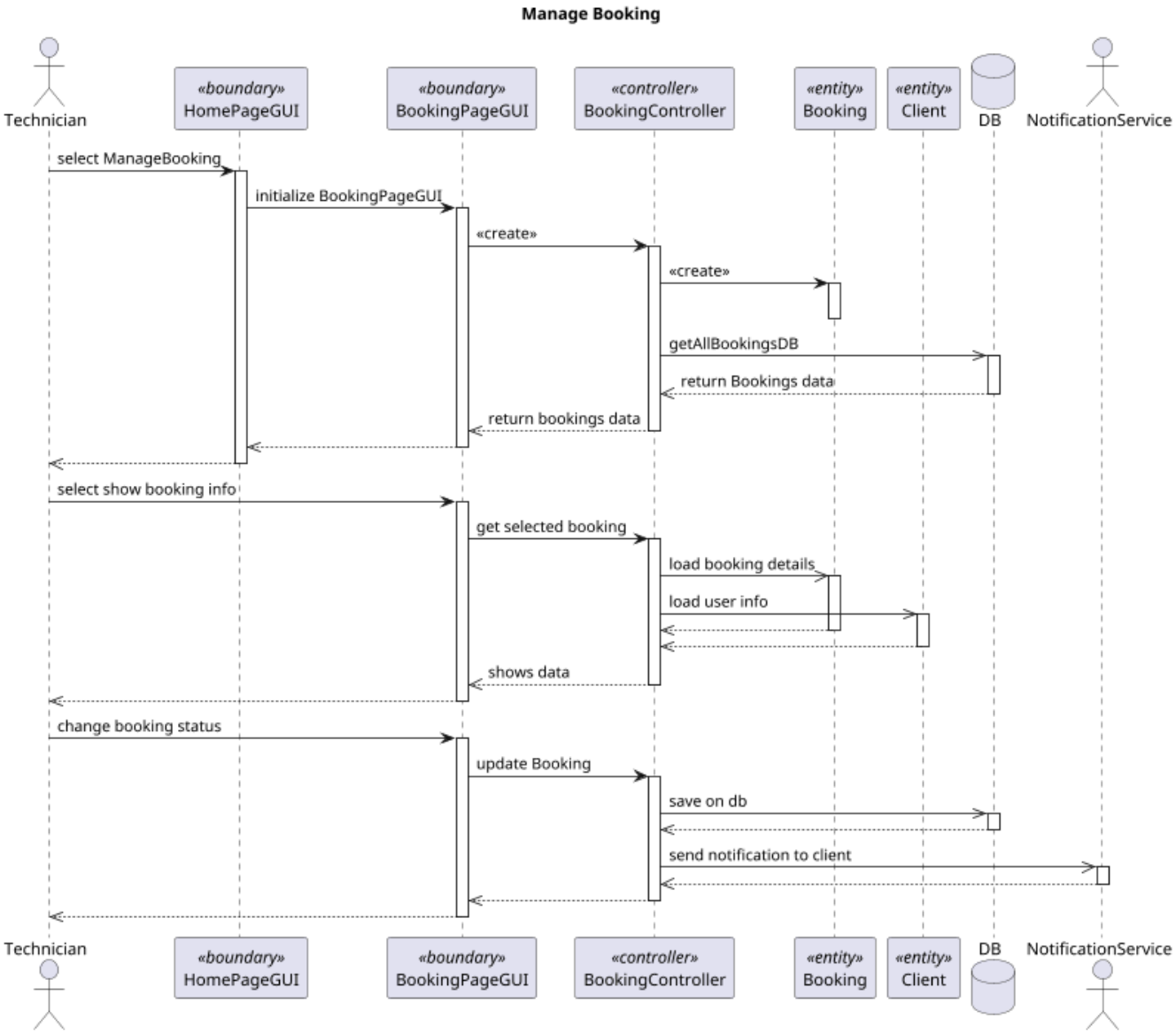


3.4. Sequence diagram

3.4.1. Booking an appointment

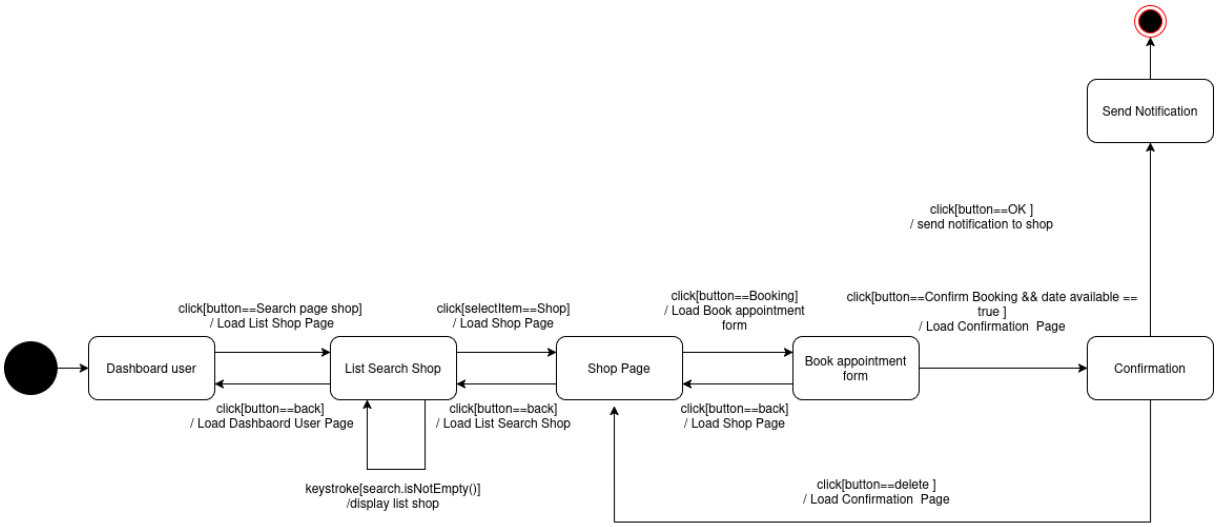


3.4.2. Manage Booking

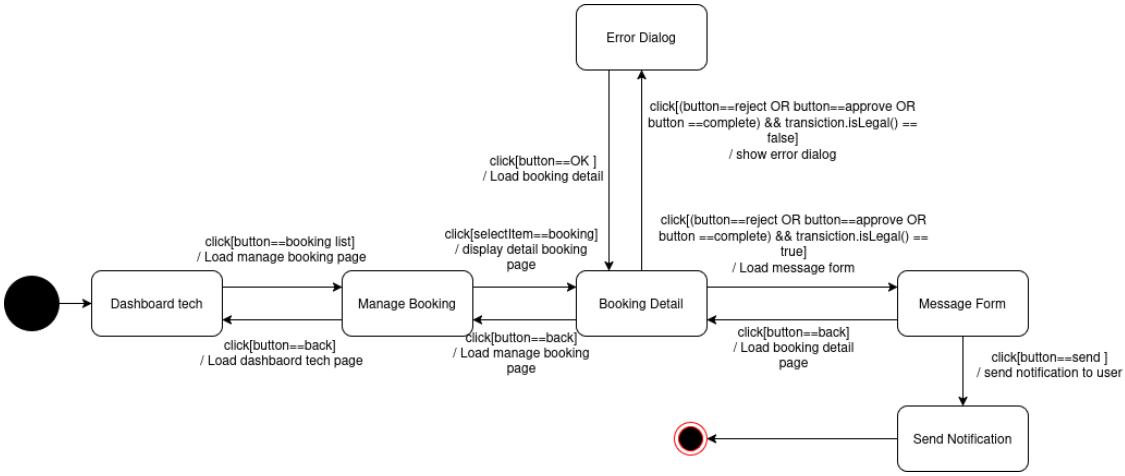


3.5. State diagram

3.5.1. Booking an appointment



3.5.2. Manage Booking



4. Testing

4.1. Booking transition from pending state - Valerio Mazza

We tested that a booking in PENDING state cannot be changed in COMPLETED state. The test verifies that the system prevents invalid state transitions.

4.2. Login with wrong role - Giulio Rustia

We tested login using technician credentials while selecting the CLIENT role and viceversa. The test verifies that authentication fails, ensuring consistency between user identity and role.

4.3. Duplicate booking prevention - Valerio Mazza

We tested that a booking cannot be inserted when the same shop, date, and timeslot already exist. The test verifies an exception is thrown, preventing duplicates.

4.4. Unique username on registration - Giulio Rustia

We tested registration with a username that already exists. The test verifies that the username is reported as unavailable, ensuring account uniqueness.

4.5. Review without completed booking - Giulio Rustia

We tested adding a review when no completed booking exists. The test verifies that an exception is thrown, ensuring only completed bookings can be reviewed.

4.6. Address update for client and shop - Valerio Mazza

We tested address updates for both a client and a technician's shop using an invalid address. The tests verify that an exception is thrown in both cases, enforcing address validation.

5. Code

5.1. GitHub Repository

GitHub JustIT

https://github.com/GiuEliNo/JustIT.git

5.2. SonarCloud

SonarCloud JustIT

https://sonarcloud.io/project/overview?id=dosti_justit-prova

6. Video Presentation

YouTube Video

https://youtu.be/V4Ah5DKxQSU